Sharded Blockchains

Matthieu Pigaglio

Thesis submitted in partial fulfillment of the requirements for

the Degree of Doctor in Applied Sciences

October 2026

ICTEAM

Louvain School of Engineering

Université catholique de Louvain

Louvain-la-Neuve

Belgium

Thesis Committee:

Pr. Wonderful **Supervisor** (Advisor)	UCLouvain/ICTEAM, Belgium
Pr. Good **Doctor**	UCLouvain/ICTEAM, Belgium
Pr. Another **Guy**	Another University, World

Sharded Blockchains

by Matthieu Pigaglio

© Matthieu Pigaglio 2019

ICTEAM

Université catholique de Louvain

Place Sainte-Barbe, 2

1348 Louvain-la-Neuve

Belgium

This work was partially supported by the F.R.S-FNRS.

Hey, I want to

tell you a TCP joke...

ANONYMOUS

Preamble

We will put here some text that will motivate the need of your thesis. Because the world needs your input, you feel invested in a four-year long quest to fill in the gap between your dreams and the reality. In particular, the following explains why the world is wrong.

Quisque consectetuer. In suscipit mauris a dolor pellentesque consectetuer. Mauris convallis neque non erat. In lacinia. Pellentesque leo eros, sagittis quis, fermentum quis, tincidunt ut, sapien. Maecenas sem. Curabitur eros odio, interdum eu, feugiat eu, porta ac, nisl. Curabitur nunc. Etiam fermentum convallis velit. Pellentesque laoreet lacus. Quisque sed elit. Nam quis tellus. Aliquam tellus arcu, adipiscing non, tincidunt eleifend, adipiscing quis, augue. Vivamus elementum placerat enim. Suspendisse ut tortor. Integer fau-

cibus adipiscing felis. Aenean consectetuer mattis lectus. Morbi malesuada faucibus dolor. Nam lacus. Etiam arcu libero, malesuada vitae, aliquam vitae, blandit tristique, nisl.

This thesis bridges this gap to achieve very interesting results. In particular, the main contributions of this thesis are the following.

- *A first very interesting contribution to highlight.*

  We here describe some lines to show what you first made. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ri-

diculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

- *A second element based on your first contribution.*

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

- *Now forget the first two contributions to see this one.*

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis

velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

- *A last point to close the loop.*

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue

est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

The thesis is organized as follows. Part I gives the background: Chapter 1 covers blockchain fundamentals, Chapter 2 the scalability problem and existing solutions, and Chapter 3 the open challenges this thesis addresses. Part II studies the locality of Ethereum transactions, Part III presents SOAPS, and Part IV presents PANDAS. Finally, the conclusion summarizes the contributions and discusses perspectives.

Bibliographic notes

Conference Publications

1. J.-L. Picard and W. Supervisor. "A First Wonderful Paper: How Did I Made It?" In: *Proceedings of the 42nd International Conference on Making the Research Great Again – MaReGA'17*. Groville, Groland, Apr. 2017, pp. 1–6. DOI: 12.3456789/ABC.2017.123456.

2. J.-L. Picard, F. Baggins, E. Aragorn, and W. Supervisor. "Unhoped: How Such a Funny Publication Got Accepted". In: *Proceedings of the 3rd Conference of the Fellowship of the Ring – FetRi'18*. Edoras, Rohan: Elven Press, 2018, pp. 161–175. ISBN: 123-4-5678-9012-3. DOI: 23.4567/8901234.5678901.

Posters and Demos

1. Legolas and J.-L. Picard. "Bowie: A Fully-Fledged Bow". In: *Proceedings of the 21st Annual International Conference on Fantastic Weapons and Demos - FaWeDe'18*. Mirkwood, Rhovanion: Elven Press, 2018, pp. 230–232. ISBN: 712-9-4787-0000-9. DOI: 80.0000/9988875.0213456.

Journal Publications

1. J.-L. Picard, J. T. Kirk, and B. Sisko. "Voyager: Killing a Wonderful Science-Fiction Story". In: *DC Communications in Computer Sciences – DC ComiCS* 13 (2017), pp. 42–78. ISSN: 9129-8239. DOI: 00.4884/w.ploins.2019.80.012.

2. B. Wayne, J.-L. Picard, and C. Kent. "Improving the Obfuscation of Research Papers with Nonsense". In: *DC Communications in Computer Sciences – DC ComiCS* 69 (2019), pp. 114–122. ISSN: 9999-8879. DOI:

00.4444/w.fulonn.2019.00.012.

Miscellaneous Contributions

1. J.-L. Picard. *On the Impact of Turboprops on the Sea Otter Reproduction.*

   Invited talk at the National Turefu – NaTure'19. Pandoran, Nov. 2019.

Acknowledgments

While this version was first brought by Quentin De Coninck, it relies on the templates of numerous previous Ph.D. students, including David Lebrun, Olivier Tilmans and Fabien Duchêne.

Contents

Preamble **i**

Acknowledgments **xi**

Table of Contents **xiii**

I Background and State of the Art **1**

1 Blockchain Fundamentals **3**

1.1 Distributed Ledgers 3

1.2 Consensus 11

1.2.1 Proof-of-Work 14

1.2.2 Proof-of-Stake 16

1.2.3 BFT Variants 18

1.3 Ethereum Proof-of-Stake 20

1.3.1 Validators 21

1.3.2 Slots and Epochs 23

1.3.3 Committees and Attestations 25

1.3.4 Fork-Choice Rule 28

1.3.5 Rewards and Safeguards 31

1.4 Transaction and State Models 34

1.4.1 UTXO Model 35

1.4.2 Account-Based Model 37

1.5 Smart Contracts and Execution Environments 41

1.5.1 The Ethereum Virtual Machine 43

1.5.2 Alternative Execution Environments 46

2 The Scalability Problem **49**

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks 50

2.2 Exploiting Workload Properties 52

2.2.1 Parallel Transaction Execution 53

2.2.2 Light Nodes and Stateless Clients 54

2.3 On-Chain Scaling: Sharding 55

2.3.1 Network Sharding 57

2.3.2 Transaction Sharding 58

2.3.3 State Sharding 58

2.3.4 Existing Sharded Blockchains and Their Limitations . 61

2.4 Off-Chain Scaling: Layer 2 64

2.4.1 Rollups 65

2.4.1.1 Optimistic Rollups 66

2.4.1.2 Zero-Knowledge Rollups 67

2.4.2 State Channels 67

2.4.3 Sidechains 68

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations 69

3 Synthesis and Motivation 73

3.1 Open Challenges 73

3.2 Common Requirements for L1 and L2 Sharding 74

3.3 Positioning of the Contributions 75

Part I

Background and State of the Art

Blockchain Fundamentals 1

1.1 Distributed Ledgers

A distributed ledger is an append-only log of transactions, which is replicated by a set of nodes that do not trust each other. Bitcoin [Nak08] was, in 2009, the first ledger of this kind to work in an open setting, and its transactions are transfers of a native currency. In 2015, Ethereum [Woo14] extended the model so that transactions can call programs (smart contracts) whose code and storage are kept in the replicated state. We describe here the elements of this model that are used in the rest of the thesis, and leave the question of how nodes agree on the content of the log to Section 1.2.

State machine replication. A ledger can be seen as a case of state machine replication [Sch90]. Each node holds a copy of a state S (in the simplest case, the balance of every account), and a transaction tx changes it through a transition function, $S' = \delta(S, tx)$. If all nodes start from the same initial state S_0, the *genesis* state, and apply the same sequence of transactions $tx_1, \ldots, tx_n$, they all obtain

$$S_n = \delta(\ldots\delta(\delta(S_0, tx_1), tx_2)\ldots, tx_n).$$

For this to hold, δ has to be deterministic. The execution environments of blockchains that support smart contracts (Section 1.5) are designed with this in mind, and give programs no access to local clocks, floating-point arithmetic, or external services, all of which could give different results on different machines. Consequently, nodes need not exchange the state to remain consistent, since it can be recomputed from the log; they only need to agree on the order of transactions, which is the role of consensus.

Blocks and hash chaining. Transactions are added to the log in batches, called blocks. A block is made of a body, which holds the ordered list of transactions, and a header with metadata. The header typically contains the hash of the parent header, a timestamp, and short commitments to the content of the block, such as the root of a Merkle tree over its transactions and, in some ledgers, a commitment to the state obtained after executing the block.

Because each header includes the hash of its parent, the blocks form a hash chain back to genesis. Linking documents through hashes to detect tampering is an older idea, used for digital timestamping [HS91]. Under the usual assumption that the hash function is collision resistant, the hash of the most recent header fixes the whole history. Changing a transaction in an old block changes the hash of that block, then the parent hash stored in its child, and so on up to the latest header, so a node that knows a recent header will notice the change. Hash chaining, however, only makes a rewrite detectable. An

attacker can still produce a different, internally consistent chain, and selecting one chain among several valid candidates is again the role of consensus (Section 1.2).

Merkle trees. These commitments are usually roots of Merkle trees [Mer87]. The leaves of a Merkle tree are hashes of data items, and every inner node is the hash of its children, so the root depends on every leaf. A proof that an item belongs to the tree consists of the sibling hashes along the path from its leaf to the root. Its size is logarithmic in the number of leaves, and anyone who knows the root can check it. The same technique applies to the state. If the header contains the root of a tree built over all state entries, such as account balances or the storage of contracts, then any single entry can be proven against that root. How the state is organized in practice depends on the transaction and state model of the ledger, which we discuss in Section 1.4.

Permissioned and permissionless ledgers. Ledgers also differ in who is allowed to maintain them. In a permissioned ledger such as Hyperledger Fabric [And+18], participants are known in advance and authenticated, and classical Byzantine fault-tolerant protocols (Section 1.2.3) can be used as they are, since these assume a fixed set of n nodes with at most f faulty ones. Public blockchains such as Bitcoin are permissionless: anyone can join or leave, and an identity is a public key that costs nothing to generate. A single participant can then create as many identities as it wants (a Sybil attack [Dou02]), and votes cannot simply be counted per identity. Voting power is instead tied to a resource that is hard to obtain, computation in proof-of-work (Section 1.2.1) or locked stake in proof-of-stake (Section 1.2.2).

This thesis only considers permissionless ledgers. For our contributions, this means that the mechanisms we add, namely the placement of state on shards in Part III and the assignment of data to nodes in Part IV, have to

work with open membership and with some malicious participants. These decisions must also be verifiable by any node, and resistant to bias from participants who would benefit from a particular outcome.

Node roles. Not all nodes do the same work [CBC22]. A full node checks every new block by re-executing its transactions and verifying that the result matches the commitments in the header. It keeps the current state, and possibly a limited window of recent states. A new full node does not usually replay the chain from genesis: it starts from a recent checkpoint, downloads the corresponding state, and validates blocks from that point on. Under proof-of-stake, the node has to obtain this checkpoint from a source it trusts, which is related to the weak subjectivity issue discussed in Section 1.2.2. An archive node is a full node that also keeps all historical states, so that it can answer queries such as the balance of an account at a given past block, at the price of a much larger disk footprint. Validators (miners under proof-of-work)

are full nodes that also propose and vote on blocks; under proof-of-stake, they are the participants that have locked stake (Section 1.2.2). Finally, light clients only follow block headers and check individual values with Merkle proofs [CBC22]. They do not re-execute transactions, and instead assume that most consensus participants are honest (Section 2.2.2).

Peer-to-peer network. Nodes are connected by a peer-to-peer overlay in which each node maintains a bounded number of connections. Transactions and blocks are propagated by gossip, where a node that receives a valid message forwards it to some of its neighbors [Dem+87]. Peers are found through a discovery protocol, often a distributed hash table such as Kademlia [MM02]. To keep the bandwidth cost bounded, practical gossip protocols send full messages only to a small set of neighbors and announce them to the others with short metadata.

The network affects the protocol because the propagation delay of blocks

bounds the rate at which blocks can be produced without degrading security [GKL15; Ger+16]. Block intervals are therefore chosen to be large compared to this delay. Part IV works at this level, since its goal is to distribute and sample data over the peer-to-peer network within the time that consensus allows.

Full replication. In the model described so far, every full node stores the whole state and re-executes every transaction of every new block. This full replication removes the need to trust other nodes, but it also implies that the capacity of the system does not increase with the number of nodes. Throughput is limited by what a single node, run by a regular participant, can download, store, and execute. Larger or more frequent blocks would raise this limit, but they would also increase the cost of running a node, thereby reducing the number of participants able to do so, and slow down block propagation, which results in more forks and weaker security [Ger+16]. Chapter 2 discusses this

problem and the main approaches to it. This thesis builds on one of them, sharding, in which each node only processes a subset of the transactions and of the data.

1.2 Consensus

As seen in Section 1.1, nodes that apply the same transactions in the same order end up with the same state. The role of the consensus protocol is to make them agree on this order, block after block, even though some of them may be faulty or malicious and messages may be delayed by the network [LSP82].

Fault and network models. Consensus protocols for blockchains generally assume *Byzantine* faults, meaning that a faulty node may behave arbitrarily. It can stop responding, send wrong messages, collude with other faulty nodes, or *equivocate*, i.e., send conflicting messages to different nodes (e.g., votes for two different blocks at the same height). Protocols also differ in what

they assume about the network. In the synchronous model, every message is delivered within a known bound Δ. In the asynchronous model, there is no such bound, and Fischer, Lynch, and Paterson showed that no deterministic protocol can then guarantee agreement if even one node may crash [FLP85]. The partially synchronous model lies in between: the network may behave asynchronously for some time, but eventually delivers messages within a bound, and agreement is possible as long as fewer than a third of the nodes are Byzantine [DLS88]. Longest-chain protocols assume a synchronous network [GKL15], whereas most BFT protocols target partial synchrony [CL99; Yin+19].

Safety and liveness. The properties expected from consensus are usually split in two. *Safety* requires that honest nodes never confirm two conflicting blocks, so that a confirmed transaction is never reverted. *Liveness* requires that the protocol keeps making progress, so that a valid transaction submitted

by an honest client is eventually included. These two properties are in tension when the network can be partitioned. A protocol that keeps producing blocks on both sides of a partition cannot also guarantee that these blocks will never conflict. Some protocols therefore combine a chain that keeps growing under any conditions with a slower finality mechanism that only confirms a prefix of this chain when enough participants agree on it [NTT21]. Ethereum follows this design, as we detail in Section 1.3.

Sybil resistance. Since votes in a permissionless ledger are weighted by a costly resource rather than counted per identity (Section 1.1), security assumptions are expressed in terms of this resource, for instance that honest participants control a majority of the computing power in proof-of-work (Section 1.2.1), or more than two thirds of the stake in proof-of-stake (Section 1.2.2). Classical Byzantine fault-tolerant protocols (Section 1.2.3) do not provide Sybil resistance by themselves, but they are often run on top of proof-

of-stake, by a committee selected according to stake.

1.2.1 Proof-of-Work

Proof-of-work was originally proposed to limit spam by requiring a small computation for every email [DN92], and Bitcoin used it to elect who appends the next block [Nak08]. To produce a valid block, a miner has to find a value (nonce) such that the hash of the block header falls below a target. The only known way to do so is to try values one after the other, so the probability that a given miner finds the next block is proportional to its share of the total computing power. The target is adjusted periodically so that blocks are produced at a roughly constant rate, every ten minutes on average in Bitcoin.

Two miners may find a block at nearly the same time, in which case the chain temporarily splits into two branches, or *forks*. Nodes resolve forks with a *fork-choice rule*. Bitcoin uses the longest chain rule, or more precisely the chain with the most accumulated work, and miners extend the branch se-

lected by this rule. GHOST [SZ15] proposed instead to follow the heaviest subtree, which also counts the blocks on discarded branches and remains secure at higher block rates. The fork-choice rules of later proof-of-stake protocols build on this idea (Section 1.3.4).

Under these rules, finality is only probabilistic. A block can always be reverted by a heavier branch, but the probability of this decreases exponentially with the number of blocks built on top of it, which is why users wait for several confirmations. The Bitcoin backbone analysis [GKL15] shows that the protocol is secure if honest miners hold a majority of the computing power and if blocks propagate quickly compared to the block interval. When blocks are larger or produced more often, more forks occur, and the computing power spent on discarded branches no longer contributes to security [Ger+16]. Throughput cannot therefore be increased simply by changing these parameters, which is the starting point of Chapter 2. Proof-of-work

also has a high energy cost, since security is proportional to the computation spent by miners.

1.2.2 Proof-of-Stake

In proof-of-stake, participants called *validators* lock an amount of currency, their *stake*, and their weight in the protocol is proportional to it. Validators are rewarded for proposing blocks and voting correctly. Ouroboros [Kia+17] and Algorand [Gil+17] are two examples with formal security proofs. Ouroboros keeps a longest-chain structure and selects block producers according to stake. Algorand selects small committees by cryptographic sortition, and each committee runs a Byzantine agreement protocol to decide on a block.

Because the stake is held by the protocol itself, misbehavior can be punished directly. If a validator equivocates (e.g., by signing two different blocks for the same position), the two signed messages are a proof of misbehavior that identifies the culprit [She+21]. Anyone can include this proof in a block,

and the protocol then destroys part of the validator's stake, which is called *slashing*. Security therefore rests on economic incentives, since an attack requires a large share of the stake and exposes that stake to slashing. We come back to rewards and slashing in Section 1.3.5, as Part III relies on them.

Proof-of-stake protocols need to select, at each step, the proposer of the next block and the members of the committees that vote on it. This selection is done using randomness that is itself produced by the protocol [Gil+17]. If participants can predict this randomness long in advance, they can prepare attacks on the selected nodes, and if they can influence it, they can increase their chances of being selected for positions that benefit them. Such manipulation has been shown to be possible in practice [Nag+25]. Our designs face the same issue, since both assign tasks to nodes based on this randomness (Section 1.3.3).

Proof-of-stake also raises some issues that do not exist with proof-of-

work. Since producing a block costs almost nothing, a validator could vote on every competing branch at no cost, the so-called *nothing-at-stake* problem, which slashing is designed to prevent. Validators that have withdrawn their stake can no longer be slashed, but their old keys can still be used to sign an alternative history starting from far in the past, which is called a *long-range attack*. As a consequence, a node joining the network cannot tell the correct chain from such an alternative using only the genesis block, and has to obtain a recent checkpoint from a source it trusts. This assumption is known as *weak subjectivity*.

1.2.3 BFT Variants

Classical Byzantine fault-tolerant (BFT) protocols, starting with PBFT [CL99], assume a fixed and known set of n nodes, of which at most f are Byzantine, with $n \geq 3f+1$. Decisions are taken by *quorums* of $2f+1$ nodes. Any two such quorums share at least $f + 1$ nodes, and therefore at least one honest node,

which never votes for two conflicting values. Two conflicting decisions can thus never both gather a quorum. In PBFT, a leader proposes a value, and the nodes agree on it in three phases (pre-prepare, prepare, and commit). If the leader is faulty or too slow, the nodes run a *view change* to replace it [CL99]. Unlike longest-chain protocols, BFT protocols give immediate finality: once a decision is taken, it is never reverted as long as the bound on f holds.

The main limitation of these protocols is the number of messages they exchange. In PBFT, every node sends its votes to every other node, so each decision requires $O(n^2)$ messages, which limits the number of participants in practice. HotStuff [Yin+19] reduces this cost to $O(n)$ per decision by having the leader collect the votes and combine them into a single threshold signature. Signature aggregation schemes such as BLS [BLS01] are used in the same way to combine thousands of votes into a single signature that can be verified at once.

Sharded blockchains such as OmniLedger [Kok+18] and RapidChain [ZMR18] run a BFT protocol inside each shard, among a committee of nodes assigned to it. Smaller committees reach agreement faster, but they are more likely to contain more than a third of malicious members by chance. We discuss this trade-off between committee size and security in Section 2.3.

1.3 Ethereum Proof-of-Stake

The contributions of this thesis target Ethereum, and they depend on how its consensus protocol works. Part III builds on its validator committees and on its penalties, and Part IV has to fit in the timing of its slots. We therefore describe it in more detail than the protocols of Section 1.2. Unless stated otherwise, the parameters given here come from the consensus specifications [Eth].

Ethereum started with proof-of-work, and switched to proof-of-stake in September 2022, in an upgrade known as *the Merge*. Since then, an Ethereum node runs two separate programs. The *consensus client* runs the *beacon chain*,

which manages the set of validators, assigns them to committees, and decides which blocks are part of the chain and which are final [Sch+22]. The *execution client* keeps the state and the pool of pending transactions, and executes the transactions of each block in the Ethereum Virtual Machine (Section 1.5.1). The two communicate through a local interface, the Engine API: the consensus client asks the execution client to build or to check the *execution payload*, which is the list of transactions carried by a beacon block. The idea of a beacon chain that coordinates a set of validators and assigns them to committees comes from earlier sharding designs, in which the beacon chain was meant to coordinate many shard chains (Section 2.3). The system model of Part III follows this design.

1.3.1 Validators

To become a validator, a participant deposits at least 32 ETH in a deposit contract and runs both a consensus and an execution client. Each validator has

an *effective balance*, derived from its actual balance, which determines both its weight in votes and its probability of being selected as proposer or aggregator. Until 2025, the effective balance was capped at 32 ETH, so that large stakeholders had to run many validators. The Pectra upgrade raised this cap to 2048 ETH, which allows them to merge their stake into fewer validators. The stake is, however, unevenly distributed, as staking pools and liquid staking services control a large share of it, even though it is split across many separate validators.

Validators have three kinds of duties. They propose a block when they are selected as the proposer of a slot. They *attest* once per epoch, i.e., they sign a vote on the current head of the chain and on the latest checkpoints (Section 1.3.3). Some of them are also selected as aggregators, and combine the attestations of their committee into a single signature. Finally, since the Altair upgrade, a *sync committee* of 512 validators, renewed every 256 epochs

(about 27 hours), signs each block header so that light clients can follow the chain without tracking the whole validator set (Section 2.2.2).

Validators enter and leave the active set through queues. The number of validators that can be activated or that can exit in each epoch is bounded by a *churn limit*, which keeps the validator set from changing too quickly, and in particular prevents an attacker from gaining a large weight in a short time. The active set is on the order of a million validators. At this scale, a classical BFT protocol in which every validator exchanges votes with every other (Section 1.2.3) is impractical, and the protocol relies instead on committees and on signature aggregation, which we describe below.

1.3.2 Slots and Epochs

Time is divided into *slots* of 12 seconds, grouped into *epochs* of 32 slots, or 6.4 minutes (Figure 1.1). Each slot has a single proposer, which may publish one block. Otherwise, the slot remains empty and the protocol proceeds to the

next one.

Proposers are selected at random, weighted by effective balance, using a value called the RANDAO mix. Each proposer includes in its block a signature over the current epoch number, and this signature is mixed into the RANDAO value. The mix obtained at the end of an epoch is used as the seed to select proposers and committees for a later epoch. The signature of each proposer is fixed in advance, so a proposer cannot choose its contribution. It can, however, withhold its block, in which case its contribution is skipped. The last proposers of an epoch can use this to choose between several possible seeds, and thus bias the selection of future proposers in their favor [Nag+25]. The selection is also known before the corresponding slots, which allows nodes to prepare, but also reveals to an attacker which node to target at a given time (e.g., with a denial-of-service attack).

Within a slot, the protocol follows a fixed schedule. At the start of the slot

(0 s), the proposer publishes its block, which is then propagated through the network. At 4 s, the validators of the committee assigned to this slot attest to the head of the chain as they see it, or earlier if they have already received and checked the block. At 8 s, aggregators publish the aggregated attestations of their committee. A validator thus has 4 seconds to receive and check a block before voting. If the block arrives late, or cannot be checked in time, the validator votes for the previous head, and the block is likely to be left out of the chain. In Part IV, validators must additionally verify that the data attached to the block is available, and this verification has to fit within the same 4 seconds.

1.3.3 Committees and Attestations

At each epoch, the active validators are shuffled using the RANDAO seed and split into committees. Each slot of the epoch receives between 1 and 64 committees, and every validator belongs to exactly one committee in the

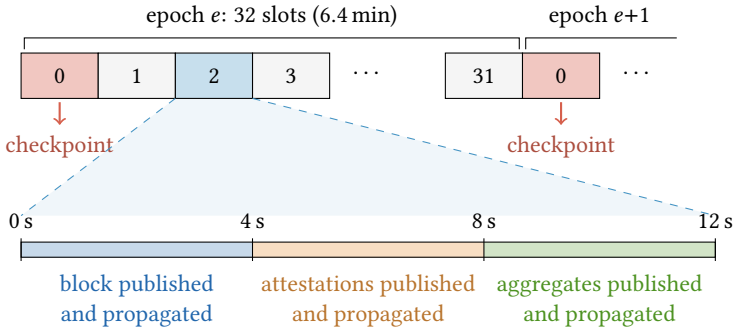


Figure 1.1: Slots and epochs in Ethereum proof-of-stake. An epoch has 32 slots of 12 seconds, and the first slot of each epoch serves as its checkpoint. Within a slot, the block is published at 0 s, attestations at 4 s, and aggregated attestations at 8 s.

epoch, so that it attests exactly once per epoch. Committees have a target size of 128 validators, a size chosen so that a committee is very unlikely to have a dishonest majority as long as the attacker controls less than a third of the stake. With around a million validators, the actual committees are much larger than this minimum.

The shuffle is deterministic and depends only on public data, i.e., the seed and the list of active validators. Any node can therefore recompute the committees and check that a given attestation comes from a member of the right committee, without trusting anyone. Both of our designs assign tasks

to nodes in the same way, with a deterministic function of public randomness that any node can recompute. In Part III, it is used to elect the validator that computes the placement of state on shards, and the witnesses that check it. In Part IV, it determines which node is responsible for which part of the data.

An attestation contains three votes. The *head vote* designates the block that the validator considers to be the head of the chain, and is used by the LMD-GHOST fork-choice rule. The *source* and *target* votes designate two checkpoints, and are used by the Casper FFG finality mechanism [Sch+22]. Attestations are signed with BLS signatures [BLS01], which can be aggregated: the signatures of all the members of a committee who cast the same vote are combined into a single signature, together with a bit field indicating who signed. Without aggregation, blocks would have to include about a million signatures per epoch, which would exceed the bandwidth available to nodes.

1.3.4 Fork-Choice Rule

The consensus protocol of Ethereum, called Gasper, combines two mechanisms [Sch+22; NTT21]. LMD-GHOST is a fork-choice rule that selects the head of the chain and allows it to grow even when many validators are offline. Casper FFG is a finality mechanism that runs on top of this chain and finalizes some of its blocks. Gasper follows the ebb-and-flow design of Section 1.2, in which the chain selected by LMD-GHOST remains available and the finalized prefix guarantees safety under network partitions [NTT21].

LMD-GHOST adapts the GHOST rule (Section 1.2.1) to votes. It starts from the latest justified checkpoint and, at each fork, follows the branch whose subtree has received the most weight, counting only the latest attestation of each validator (hence *latest message driven*).

Casper FFG works at the level of epochs. The checkpoint of an epoch is the block at its first slot. When validators holding at least two thirds of the

stake attest with the same source and target, the target checkpoint becomes *justified*. A justified checkpoint becomes *finalized* when the checkpoint of the next epoch is justified with it as source. In normal conditions, a block is therefore finalized after two to three epochs, or about 13 to 19 minutes. Reverting a finalized block would require finalizing two conflicting checkpoints. This is only possible if at least a third of the stake signs contradictory votes, and such votes can be proven and slashed (Section 1.3.5).

Several attacks on this fork-choice rule have been identified. In a balancing attack, a small number of Byzantine validators release their votes at chosen times so that honest validators remain split between two branches. In an ex-ante reorganization, an attacker withholds a block and its votes to later outweigh the next honest block [Sch+22]. To mitigate these attacks, the protocol now uses *proposer boost*: a block received on time, before the 4 s mark, temporarily receives additional weight in the fork choice, equal to a fraction

of the weight of a committee.

Validators are expected to vote only for blocks that are valid and whose data is *available*, meaning that anyone can download it. With Data Availability Sampling, introduced in Section 2.4, validators check availability by downloading a few random samples of the data instead of all of it. Under the *tight* fork-choice rule, a validator performs this check before attesting, so that a block whose data cannot be retrieved never becomes the head, even if it is otherwise valid. Under the *trailing* fork-choice rule, the check is performed later, which gives more time but weaker guarantees, since a block may be built upon before its data has been checked. Part IV targets the tight fork-choice rule, which is why sampling has to be done within the first 4 seconds of the slot [Pig+25].

1.3.5 Rewards and Safeguards

Validators are paid for performing their duties correctly and on time, and lose a small amount when they miss them. The reward for an attestation depends on whether each of its three votes is correct and on how quickly it was included in a block. Proposers are rewarded for including attestations and evidence of misbehavior in their blocks, and members of the sync committee for signing headers.

A validator is *slashed* when it signs messages that contradict each other. There are three such cases: proposing two different blocks for the same slot, casting two attestations with the same target epoch but different content, and casting an attestation whose source and target surround those of another of its attestations [Sch+22]. In each case, the proposer that includes the resulting proof (Section 1.2.2) receives a small reward. A slashed validator immediately loses a small part of its balance, is forced to exit, and can only withdraw its re-

maining funds after about 36 days. In the middle of this period, it pays an additional *correlation penalty*, which is proportional to the total stake slashed in the same time window. An isolated fault, typically caused by a misconfiguration, therefore incurs a small penalty, whereas a coordinated attack involving a third of the stake leads to the loss of the entire stake of the attackers.

A different safeguard handles the case where finality stops. If more than a third of the stake is offline, no checkpoint can gather two thirds of the votes, and the chain stops finalizing. After a few epochs without finality, the *inactivity leak* starts: the balances of the validators that do not participate decrease, faster and faster, until the remaining active validators again hold two thirds of the stake and finality resumes. These penalties must, however, be designed carefully: Pavloff et al. show that Byzantine validators can exploit them to reduce the stake of honest validators and increase their own relative weight [PAT24].

Ethereum thus does not prevent misbehavior, and instead penalizes it when it can be proven and attributed to a validator. SOAPS (Part III) relies on the same mechanisms (election, stake, and slashing) instead of a separate consensus protocol. For this reason, any deviation from the SOAPS protocol must be detectable and attributable to a specific validator, and its challenge mechanism serves this purpose.

Finally, the block of each slot is not always built by its proposer. The order of transactions in a block can be a source of profit (e.g., by placing one's own transaction just before or after a large trade), and this profit is called *maximal extractable value* (MEV) [Dai+20]. Extracting MEV requires specialized infrastructure, and Ethereum has therefore adopted, outside of the protocol, a form of *proposer-builder separation* (PBS). Specialized *builders* assemble blocks and bid for the right to have them proposed, and *relays* forward the highest bid to the proposer, which signs the block header without seeing its content. A small

number of builders produce most of the blocks [Hei+23]. As a result, the data distributed and sampled in Part IV initially comes from a few well-connected and well-provisioned builders, whereas most of the nodes that sample it run on modest hardware.

1.4 Transaction and State Models

The state machine view of Section 1.1 leaves open what the state is made of and how a transaction designates the part of the state it reads and modifies. The state model defines which data a transaction accesses, and therefore when two transactions conflict. It determines whether transactions can be executed in parallel (Section 2.2.1) and how the state can be divided between shards (Section 2.3.3). The *unspent transaction output* (UTXO) model, introduced by Bitcoin [Nak08], represents the state as a set of coins, while the *account-based* model, used by Ethereum [Woo14], represents it as a set of accounts with balances and storage. This thesis works with the account-based

model, but we first describe the UTXO model, as the comparison explains several of the difficulties addressed in Parts II and III.

1.4.1 UTXO Model

In the UTXO model, the state is the set of transaction outputs that have not been spent yet. Each output holds an amount and a condition that must be satisfied to spend it, usually a signature from the owner's key. A transaction consumes one or more existing outputs, its *inputs*, and creates new ones. Each input refers explicitly to the output it spends, by the identifier of the transaction that created it and its position in that transaction, and provides the data that satisfies its spending condition. The total amount of the new outputs cannot exceed that of the inputs, and the difference is paid as a fee to the block producer (Figure 1.2). An output can only be spent once. Two transactions that spend the same output are a *double spend*, and at most one of them can be included in the ledger.

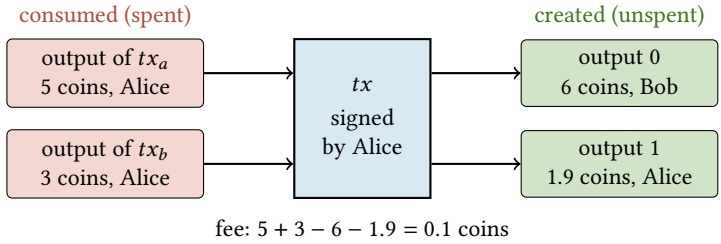


Figure 1.2: A transaction in the UTXO model. It consumes two outputs owned by Alice, which are removed from the set of unspent outputs, and creates two new ones: a payment to Bob and the change returned to Alice. The difference between inputs and outputs is the fee.

Since inputs are listed in the transaction, the part of the state a transaction depends on is known before it is executed. Two transactions conflict only if they spend the same output, or if one spends an output created by the other. Transactions that do not share outputs can be validated in parallel, and a sharded design can place each output on a shard (e.g., according to the identifier of the transaction that created it) and know in advance which shards a transaction involves. For the same reason, several of the first sharded blockchains were designed for the UTXO model [Kok+18; ZMR18], and transaction placement methods have been proposed for it [Ngu+19].

The limitation of this model is that outputs are consumed as a whole and

do not hold any persistent data. Applications that maintain shared, long-lived state, such as a token or an exchange used by many users, are therefore difficult to express. Extended UTXO models add data and more expressive scripts to outputs while keeping explicit inputs, but they require the application to split its state into outputs that are consumed and recreated at every update. Since the workloads we study and the systems we target use accounts, we do not consider the UTXO model further.

1.4.2 Account-Based Model

In the account-based model, the state is a mapping from addresses to accounts. In Ethereum, there are two types of accounts. *Externally owned accounts* (EOA) are controlled by a private key, and hold a balance and a *nonce*, which counts the transactions sent by the account. Each transaction carries the nonce of its sender, which forces the transactions of an account to be executed in order and prevents the same signed transaction from being replayed.

Smart contract accounts (SCA) are created by deploying code. In addition to a balance and a nonce, they hold this code and a persistent *storage*, a mapping from 256-bit keys, called slots, to 256-bit values, which only the contract's code can modify (Section 1.5). A token contract, for instance, typically keeps the balance of each of its holders in a separate storage slot.

A transaction is sent by an EOA. It either transfers currency to another account, deploys a new contract, or calls an existing contract with some input data. The called contract may in turn call other contracts, which may call others, so that a single transaction can read and modify many accounts and storage slots. The block only contains the original transaction, and the resulting *internal transactions* can only be observed by executing it (Section 1.5.1) [PKR23]. Interactions are also very unevenly distributed, as a small number of accounts, such as exchanges, popular token contracts, and decentralized finance protocols, are involved in a large share of all transactions [GDW19].

The main difference with the UTXO model is that the state accessed by a transaction is not known in advance. It depends on the code of the contracts involved and on the current values in their storage, which may change between the moment the transaction is signed and the moment it is executed. The same call can thus access different slots, or different contracts, depending on the transactions executed before it. As a consequence, whether two transactions conflict, meaning that one writes data that the other reads or writes, is in general only known after they have been executed. Ethereum lets a transaction declare an *access list* of the accounts and slots it intends to use, but the list is optional, and the transaction can still access state that it does not list. Parallel execution (Section 2.2.1) and state sharding (Section 2.3.3) are therefore harder with accounts than with UTXOs. Part II measures how the accesses of real transactions are distributed over time and across transactions, and the same uncertainty complicates the placement of accounts on shards in

Part III.

Executing a transaction also has a cost, measured in *gas*. Each operation of the execution environment has a fixed gas cost, which reflects the resources it uses, and accessing or writing storage is among the most expensive operations. The sender sets a maximum amount of gas for its transaction, and pays for the gas actually used. Since the London upgrade, the price of gas has two components [Rou24]. A *base fee*, set by the protocol, increases when blocks are fuller than a target and decreases when they are emptier, and is burned. A *priority fee*, chosen by the sender, goes to the block proposer and determines which transactions it includes first when demand is high, and therefore how long a transaction waits before being included [Pac+23]. Each block also has a *gas limit*, which bounds the total amount of computation it may contain. In effect, the gas limit sets the throughput of the chain. It is kept low enough for nodes with modest hardware to execute each block within a slot, as a

consequence of the full replication discussed in Section 1.1.

1.5 Smart Contracts and Execution Environments

A *smart contract* is a program stored in the ledger, whose code is executed by every full node whenever a transaction calls it. Its state, such as balances or open positions, is part of the replicated state, and can only be modified by the contract's own code. Since all nodes execute the same code on the same state, a contract call produces the same result on every node, and no party needs to be trusted to execute it correctly.

A large part of the activity on Ethereum involves a few types of contracts. Token contracts keep track of who owns which units of a fungible asset, following the ERC-20 interface, or of non-fungible items, following ERC-721. Decentralized exchanges let users swap one token for another against pools of liquidity held by a contract. Lending protocols let users borrow tokens against collateral, and bridges lock assets on one chain to release equivalent

assets on another. In the workload we analyze in Part II, transactions involving smart contracts have become dominant over time, whereas simple transfers, which represented more than 80% of early transactions, have become marginal [PKR23].

Contracts can call each other, and applications are often built by composing existing contracts. A swap on a decentralized exchange, for instance, may go through a router contract, then through one or several pools, each of which calls the token contracts of the assets it holds, so that a single transaction can involve tens of contracts. Figure 1.3 shows a simplified example, in which a two-step swap already involves six contracts.

These calls create the dependencies between accounts in the account-based model (Section 1.4.2). They also explain why some contracts, such as widely used tokens or exchange pools, are accessed by a large fraction of all transactions, and become contention points when transactions are executed

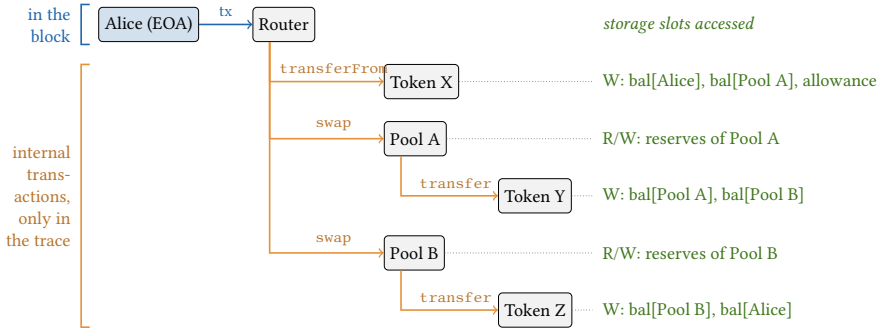


Figure 1.3: Calls made by a transaction in which Alice swaps token X for token Z through two exchange pools (X to Y in Pool A, then Y to Z in Pool B). Only the transaction from Alice to the router is recorded in the block. The internal transactions, and the storage slots that each contract reads (R) and writes (W), are only visible by executing the transaction.

in parallel or when the state is split between shards.

1.5.1 The Ethereum Virtual Machine

Ethereum contracts are executed by the Ethereum Virtual Machine (EVM) [Woo14].

The EVM is a stack machine that operates on 256-bit words. During a call, a contract has access to a stack, a temporary *memory* that is discarded at the end of the call, the input data of the call, and its persistent *storage* (Section 1.4.2). Each instruction has a gas cost, and execution stops with an error when the gas provided by the transaction runs out, so that every execution

is guaranteed to terminate. Storage accesses are by far the most expensive instructions. Reading a storage slot for the first time in a transaction costs hundreds of times more gas than an arithmetic operation, and writing a slot that was previously zero costs even more, because these operations require accessing and updating the state database of every node.

A contract calls another contract with the CALL instruction, which runs the code of the callee on the callee's storage. STATICCALL does the same but forbids any modification of the state, and DELEGATECALL runs the code of the callee on the storage of the caller, which is how libraries and upgradable contracts are implemented. If a call fails, its changes to the state are reverted, but the caller can continue and decide how to handle the failure. Transactions in a block are executed one after the other, in the order chosen by the block producer, and each transaction sees the state left by the previous one.

The state is stored in Merkle Patricia Tries, which combine the Merkle

trees of Section 1.1 with a prefix tree (Patricia trie), so that each key determines a unique path from the root to its value. The state is organized in two levels. The *world-state trie* maps the hash of each address to the corresponding account, which contains its nonce, balance, code hash, and storage root. Each contract has its own *storage trie*, which maps the hash of each storage slot to its value. The root of the world-state trie is the state root included in each block header. Reading a value requires following a path from the root of the trie down to a leaf, which can take several disk accesses. Since keys are hashed, entries that are related in the application, such as two slots of the same contract, end up in unrelated parts of the trie and on disk. Accessing the state is therefore one of the main costs of executing a block, and clients rely on caches and on flat copies of the latest state to reduce it. Such caches are only effective if transactions tend to access state that was accessed recently, which is the temporal locality we measure in Part II.

The read and write sets of a transaction cannot be obtained from the block alone, since internal transactions are not recorded in it (Section 1.4.2). To obtain them, we replay the whole chain with the Go Ethereum (geth) client and record, for each transaction, the accounts it reads and writes, including those accessed by its internal transactions (Figure 1.3). Replaying the chain up to the Merge takes about three days and produces a dataset of about 2 TB, and Part II describes the tool we built to process it [PKR23].

Sequential execution over a single global state, combined with the block gas limit that keeps execution within reach of modest nodes (Section 1.4.2), caps Ethereum's throughput to roughly 15 to 30 tx/s.

1.5.2 Alternative Execution Environments

Many other blockchains, as well as most layer-2 protocols (Section 2.4), reuse the EVM. Being EVM-compatible allows them to run existing contracts and to reuse the same development tools, and their workloads tend to resemble that

of Ethereum, since they host the same kinds of applications. Other chains use different virtual machines (e.g., based on WebAssembly) but keep the same model of contracts executed sequentially over shared state.

A different line of work changes the execution model to allow parallel execution. Dickerson et al. proposed that block producers execute contract calls speculatively in parallel, and publish the resulting schedule so that validators can replay the block in parallel and deterministically [Dic+17]. Several recent blockchains were designed with parallel execution in mind. In Solana, each transaction declares the accounts it will read and write, and the runtime executes in parallel the transactions whose accounts do not overlap. Sui represents the state as objects, and distinguishes objects owned by a single user from shared objects. Transactions that only use owned objects do not need to be ordered with respect to others and can bypass consensus, and only transactions on shared objects go through consensus [Bla+24]. Aptos uses

Block-STM, which executes the transactions of a block optimistically in parallel, detects conflicts at runtime, and re-executes the transactions involved, so that the result is the same as a sequential execution in the order of the block [Gel+23].

These designs follow two approaches. Either transactions declare their dependencies in advance, which is easy in the UTXO model but constrains how contracts are written in the account-based model, or the system executes optimistically and assumes that conflicts are rare. In both cases, the benefit depends on the workload: transactions that access the same accounts must still be executed sequentially. Section 2.2 discusses these techniques further, and Part II measures to what extent real transactions allow them.

The Scalability Problem 2

- Here I explain why blockchains do not scale, present the two families of solutions (sharding on L1, and L2s), and for each one say clearly what is still missing.

- Much of this can come from the related work sections of the locality, SOAPS and PANDAS papers, extended.

- Order: bottlenecks, optimizations that rely on the workload, sharding, L2 and data availability, and the limitations of each.

- At the end, a table comparing existing approaches on a few criteria: what gets sharded, whether it uses the workload, whether assignments can be verified, whether it is part of consensus, and whether it was

evaluated on a real workload.

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks

- Throughput is limited because every node re-runs every transaction. Compare with centralized payment systems to give an order of magnitude [Ger+16].
  - Bitcoin and Ethereum L1 handle tens of transactions per second, card networks thousands.
  - Bigger blocks or a higher gas limit mean more expensive nodes and slower propagation [Ger+16].
- Latency depends on the block interval and on how long finality takes, both limited by propagation and consensus rounds.
  - On Ethereum a transaction gets in within a 12 s slot, and is final after about two epochs, so around 13 minutes [Pac+23].

- Running a full node costs storage (the state keeps growing), bandwidth (blocks and data) and CPU. If these go up, small participants drop out and the network becomes less decentralized [Ger+16].
  - The state already holds hundreds of millions of accounts, and the history is much bigger.
  - For publishing data, bandwidth is what runs out first. This is the motivation for Part IV.
- I can mention the trilemma here, but only briefly, as context. It is not what drives the thesis.
- The message I want to get across: L1 and L2 both need to scale, and the general answer is sharding, meaning that no single node handles every transaction or every piece of data.
  - Sharding computation means not every node executes every transaction (Part III).

  - Sharding data means not every node downloads every piece of data (Part IV).

2.2 Exploiting Workload Properties

- Some optimizations leave the protocol alone and instead count on the workload behaving well: few conflicts, and state that gets reused.

- I introduce them here because Part II checks whether those assumptions actually hold on Ethereum.

- Introduce the two kinds of locality informally (Part II gives the formal definitions):

  - spatial locality, or disjoint access parallelism: transactions running at the same time touch different state;

  - temporal locality: state used recently is likely to be used again soon.

2.2.1 Parallel Transaction Execution

- The idea is to run transactions that do not conflict at the same time inside a block.
  - The result must still be the same as running them one by one in block order.
- Three ways to do it: declare access lists up front (EIP-2930, Solana), run optimistically and redo on conflict (Block-STM [Gel+23]), or have the proposer compute a schedule (Dickerson et al. [Dic+17]).
  - With declared access lists, declaring too much loses parallelism and declaring too little makes the transaction fail.
  - The optimistic approach needs nothing from users, but it slows down when there are many conflicts [Gel+23].
  - In Dickerson et al., the proposer runs the block speculatively and

publishes a schedule that validators can replay in parallel [Dic+17].

- How well any of this works depends on how many transactions conflict, which is the spatial locality measured in Part II.

- I do not contribute to parallel execution myself. I mention it because Part II's results say something about it, and I come back to it only as a perspective.

2.2.2 Light Nodes and Stateless Clients

- Light clients check headers and Merkle proofs instead of keeping the whole state [CBC22].

  - They trust that the majority of the consensus (in Ethereum, of the sync committee) is honest. On their own they cannot spot an invalid block or missing data [CBC22].

  - Fraud proofs and data availability sampling were proposed to fix

that [Al-+21]. This leads into Section 2.4.4.

- Stateless clients go further: blocks come with witnesses for the state they touch, and caching recently used state makes those witnesses smaller.
  - Witness size costs bandwidth, and the trie design (Merkle Patricia vs. Verkle or binary trees) changes how big the proofs are.
- How well this works depends on how often state is reused, which is the temporal locality measured in Part II.

2.3 On-Chain Scaling: Sharding

- The principle: split nodes, state and transaction processing over several chains running in parallel (shards), each with its own committee, and coordinate them through a beacon chain [Wan+19].
  - In the ideal case, throughput grows linearly with the number of shards.

  - The beacon chain keeps the validator list and the randomness, assigns validators to shards, and records checkpoints (cross-links) of shard blocks.

- Security: a small committee is easier to take over, which is why nodes are assigned at random and reshuffled regularly [Ava+23].

  - Show how the chance of a corrupted committee depends on committee size and on the overall share of malicious nodes (hypergeometric tail). Committees are made big enough for this to be negligible.

  - Avarikioti et al. give a formal definition of what robust sharding should guarantee, including against adaptive adversaries [Ava+23].

- Following Wang et al. [Wan+19], split the discussion into network, transaction and state sharding.

- Reuse the SOAPS figure that shows a sharded blockchain with an intra-

shard and a cross-shard transaction.

2.3.1 Network Sharding

- This is about splitting nodes into committees (node placement).
  - A node then only takes part in consensus and gossip for its own shard.
- Assignment uses randomness that can be verified: RandHound in OmniLedger [Kok+18], the Cuckoo rule in RapidChain [ZMR18], RANDAO in Ethereum [Nag+25]. The goal is to resist Sybil and adaptive attackers.
  - Elastico creates identities with PoW puzzles [Luu+16], while PoS designs use stake.
  - Reshuffling every epoch leaves an adaptive attacker little time to corrupt a committee, but every move forces nodes to download

the state of their new shard.

2.3.2 Transaction Sharding

- Transactions are usually sent to the shard of the sender or caller [Kok+18; ZMR18].
  - It is simple, everyone knows the rule, and clients need no extra information.
- Each shard has its own mempool. Other routing rules have been proposed but are uncommon.
  - Where a transaction goes only matters together with where its state lives: it is cheap only if all its state is in the same shard.

2.3.3 State Sharding

- Each shard keeps only part of the state. Where each account lives must be known by everyone and the load must be balanced.

  - Everyone needs to know where an account lives in order to route transactions to it.
  - No shard should become the bottleneck, either for storage or for processing.

- Most designs hash the account address modulo the number of shards: Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], and the Ethereum sharding proposals.
  - No metadata to store and a uniform spread, but it completely ignores which accounts talk to each other.

- Intra-shard vs. cross-shard transactions. Cross-shard ones need an atomic commit across shards, typically two-phase commit (lock, vote, commit or abort), which costs extra consensus rounds and messages [Kok+18].
  - OmniLedger (Atomix): the client locks inputs in each shard, then unlocks or commits [Kok+18].

  - RapidChain splits a cross-shard transaction into sub-transactions that go through the output shard [ZMR18].

  - Monoxide uses relay transactions and only gets eventual atomicity [WW19]. BrokerChain uses broker accounts whose state is spread over several shards [Hua+22].

  - Whatever the protocol, you pay extra latency and messages, state stays locked for a while, and some transactions abort.

- With hash placement, most Ethereum transactions become cross-shard. Part III gives the actual number.

  - Quick intuition: with k shards, two random accounts land in the same shard with probability $1/k$, and a transaction touching more accounts is almost never local.

2.3.4 Existing Sharded Blockchains and Their Limitations

- Go through the main designs: OmniLedger [Kok+18], RapidChain [ZMR18], Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], with Wang et al.'s survey [Wan+19] as a starting point. Then tell how Ethereum dropped execution sharding and moved to rollups plus data sharding [Pig+25].
  - Put them in a table: node placement, state placement, transaction placement, cross-shard protocol.
  - Some sharded chains are running outside Ethereum (NEAR, MultiversX). Mention them without going into detail.
- First problem: hash placement does not look at the workload at all, so the cross-shard ratio is very high [Kró+21].
- A lot of work tries to place state better and cuts the number of cross-shard transactions:

  - The idea comes from databases. Schism partitions tuples with METIS on the graph of what is accessed together [Cur+10; KK98], and Mehta et al. re-shard adaptively in an untrusted setting [Meh+25].
  - Graph-based placement: TxAllo [ZPY23], BrokerChain [Hua+22]. Community detection is also common.
  - Learning-based: Spring uses deep reinforcement learning [Li+24].
  - At the transaction level, OptChain picks a shard for each transaction to avoid cross-shard ones [Ngu+19].
  - They all report big drops in the cross-shard ratio on Ethereum traces.

- Second problem: they give an explicit location to *every* account. At Ethereum's scale that is hundreds of MB of metadata that every node must keep in sync.

- Third problem: a single coordinator decides the placement, or the paper

does not say who does. This opens the door to centralization and to people optimizing for themselves, and none of it is tied to consensus.

  - Usually nobody says who runs the optimizer, on what input, or how the others can check the result.
  - Whoever controls placement could favour their own accounts (cheaper, faster) or put victims in the same shard to attack them.

- Another line of work adjusts things at runtime, moving accounts around (Król et al. [Kró+21], Huang et al. [HLZ24], BIND [Qi+26]) or scheduling transactions together (Concordia [Liu+26]). They pay for it in migration or scheduling overhead.

  - Migration locks state while it moves [HLZ24]. BIND manages to keep processing transactions during the move [Qi+26].
  - Concordia has shards cooperate when scheduling to avoid cross-shard conflicts [Liu+26].

- What is missing is a placement that stays small, that anyone can check, and that is part of consensus. That is what Part III does.

2.4 Off-Chain Scaling: Layer 2

- The principle: run transactions off L1 but keep some or all of its security, using L1 for settlement and for publishing data [Gud+20].
  - Gudgeon et al. sort them into channels, sidechains/commit-chains and rollups [Gud+20].
  - A good way to compare them is to ask what an honest user needs to get their funds back if the operator cheats.
- L2s do not make L1 scaling unnecessary. They move the pressure onto publishing data on L1.
  - That is Ethereum's "rollup-centric" roadmap: L1 becomes the place where rollups settle and publish their data.

2.4.1 Rollups

- A sequencer orders and executes transactions off chain, then posts compressed transaction data and state roots to L1 [Gud+20].
  - A bridge contract on L1 holds the deposited funds and receives the state roots.
  - Today the sequencer is usually a single operator. Users can still force their transaction in through L1 if it censors them.
- Everything relies on the data being *available*: anyone must be able to rebuild the L2 state from it [Al-+21].
- Use one concrete example, Optimism (or Arbitrum [Kal+18]). Keep it short here and go into detail in Part IV.
  - In Optimism, the sequencer makes L2 blocks, a batcher posts them to L1 (as blobs now), and a proposer posts output roots. Anyone

can rebuild the L2 chain from L1 data alone.

  - A figure would help: L2 users, sequencer, batches/blobs on L1, verifiers rebuilding the state.

2.4.1.1 Optimistic Rollups

- State roots are accepted unless someone challenges them with a fraud proof during a challenge window of about 7 days [Kal+18; Gud+20].
  - In Arbitrum the two parties bisect the disputed execution until only one instruction is left, which L1 then checks [Kal+18].
  - The price of being optimistic is the long withdrawal delay.
- Challengers need the data to recompute the disputed state.
  - If the data is withheld, nobody can write a fraud proof, and an invalid root goes through.

2.4.1.2 Zero-Knowledge Rollups

- Each batch comes with a validity proof (SNARK/STARK) checked on L1. Finality is fast, but proving is expensive [Gud+20].
  - Proving EVM execution is costly, and zkEVMs give up some compatibility to bring that cost down.
- They still need data availability, so that users can rebuild the state and withdraw.
  - A valid proof tells you the new state is correct, not what it contains. Users still need the data to know what they own.

2.4.2 State Channels

- A fixed group of participants exchange updates off chain and settle on chain at the end, as in the Lightning Network [Gud+20].

  - The channel is opened on chain, signed state updates are exchanged off chain, and it is closed on chain. In a dispute, the latest signed state wins.
  - Payment networks route a payment across several channels.

- Limitations: funds are locked, participants have to stay online, and the set of participants is fixed, so channels are not general-purpose.

2.4.3 Sidechains

- These are separate chains with their own consensus, connected to L1 by a bridge (e.g., Polygon PoS) [Gud+20].
  - Two-way peg: lock assets on L1 to mint them on the sidechain, and the other way round.
- They do not get L1's security, and bridges have been hacked many times.

  - Funds are only as safe as the sidechain validators or the bridge committee.

- Use the comparison with rollups to explain why Ethereum's roadmap went with rollups.

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations

- The data availability problem: how do you check that published data can be retrieved without downloading all of it [Al-+21]?

  - You cannot prove that someone withheld data, so you cannot slash for it. This is what makes it different from the other faults seen so far.

  - Light clients need availability guarantees for fraud proofs to be useful [Al-+21].

- EIP-4844 (blobs) [Pig+25] gives rollups their own cheaper, temporary space for data. But every node still downloads every blob, so capacity is capped by node bandwidth.
  - Blobs are large pieces of data attached to blocks, committed with KZG, not readable by the EVM, and deleted after a few weeks.
  - They have their own fee market and a small target and maximum number per block.
- Data availability sampling: erasure-code the data and let each node sample a few random pieces. This gives probabilistic guarantees while each node downloads only a small part (details in Part IV) [Al-+21; KZG10].
  - With erasure coding, any large enough fraction of the data is enough to rebuild all of it, so an attacker has to hide a lot to cause harm.

  - If that much is hidden, a few random samples are enough to notice with high probability.
  - Then explain the two ways to make sure the encoding is correct: fraud proofs [Al-+21], or KZG commitments, which rule out bad encoding by construction [KZG10].

- There are also separate DA layers (Celestia, EigenDA, Avail), with their own trust assumptions and timing.
  - A rollup that uses them no longer gets Ethereum's guarantees on its data.

- On the networking side, the options are GossipSub for dissemination and a DHT (Kademlia) for retrieval. PeerDAS is the first deployed step.
  - With gossip you need one topic per column or row. That does not scale well with many topics, and duplicate messages waste bandwidth.

  - DHT lookups take several hops, which is too slow for 4 s, and DHTs are exposed to Sybil and eclipse attacks.
  - PeerDAS has nodes keep a subset of columns and sample from their peers, but at a much smaller scale than the full design.

- What is missing: a way to spread and sample the data before the 4 s attestation deadline, at full Danksharding scale, while coping with faulty nodes and nodes that see different views of the network.

- This is what Part IV addresses [Pig+25].

Synthesis and Motivation 3

- A short chapter, a few pages, to close Part I: go from the limitations of Chapter 2 to the research questions.

3.1 Open Challenges

- C1: the designs above assume the workload has good locality, but this is rarely checked on real, recent data. This is Part II [PKR23].
  - Sharding, parallel execution and stateless clients are mostly evaluated on synthetic or short traces.
  - The question: can solid assumptions about the near future be drawn from historical data?
- C2: state placement has to cut cross-shard transactions while keeping

its metadata small, staying verifiable, and not letting anyone tilt it in their favour. This is Part III.

  - It should fit into the existing consensus and its safeguards, without a trusted coordinator.

- C3: L2 data availability has to be checked by sampling, within consensus timing, over an open P2P network. This is Part IV.

  - The constraints: a hard 4 s deadline, tens of thousands of nodes, some of them faulty or unresponsive, and no new consensus step.

3.2 Common Requirements for L1 and L2 Sharding

- Verifiable: any node must be able to recompute who is assigned what (accounts to shards, cells to nodes), with no trusted third party.

  - This uses the same trick as Ethereum committees: deterministic functions of public randomness and public state (Section 1.3.3).

- Flexible: able to adapt in large, changing systems (the workload evolves, nodes join and leave, some fail).
  - Fully deterministic schemes are too rigid, and fully ad-hoc ones cannot be checked. My designs try to sit in between.
- Decentralized: no heavy new consensus step, low cost per node, anyone can participate.
- Practical: evaluated on real Ethereum workloads and timing, and compatible with the existing protocols.

3.3 Positioning of the Contributions

- SOAPS shards computation on L1. It removes contention on execution and state, and it benefits from knowing the workload (Part II).
- PANDAS [Pig+25] shards the L2 data. It removes the bandwidth bottleneck for data availability, and it barely depends on the workload.

- The two complement each other rather than compete: a sharded L1 still has to host L2 data, and an L1 full of rollups still gains from parallel execution.

- A figure would help: L1 vs. L2 on one axis, computation vs. data on the other, with Parts II to IV placed on it.

- End with a short roadmap of Parts II to IV.

Bibliography

[Al-+18] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis. "Chainspace: A Sharded Smart Contracts Platform". In: *NDSS*. 2018.

[Al-+21] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi. "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2021, pp. 279–298.

[And+18] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, et al. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *ACM EuroSys*. 2018.

[Ava+23] Z. Avarikioti, A. Desjardins, L. Kokoris-Kogias, and R. Wattenhofer. "Divide & scale: Formalization and roadmap to robust sharding". In: *International Colloquium on Structural Information and Communication Complexity*. Springer. 2023, pp. 199–245.

[Bla+24] S. Blackshear et al. "Sui Lutris: A Blockchain Combining Broadcast and Consensus". In: *ACM CCS*. 2024, pp. 2606–2620.

[BLS01] D. Boneh, B. Lynn, and H. Shacham. "Short Signatures from the Weil Pairing". In: *Advances in Cryptology (ASIACRYPT)*. 2001, pp. 514–532.

[CBC22] P. Chatzigiannis, F. Baldimtsi, and K. Chalkias. "SoK: Blockchain Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 615–641.

[CL99] M. Castro and B. Liskov. "Practical Byzantine Fault Tolerance". In: *USENIX OSDI*. 1999, pp. 173–186.

[Cur+10] C. Curino, E. P. C. Jones, Y. Zhang, and S. R. Madden. "Schism: a workload-driven approach to database replication and partitioning". In: *Proc. VLDB Endow.* (2010).

[Dai+20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability". In: *IEEE S&P*. 2020, pp. 910–927.

[Dem+87] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry. "Epidemic Algorithms for Replicated Database Maintenance". In: *ACM PODC*. 1987, pp. 1–12.

[Dic+17] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen. "Adding Concurrency to Smart Contracts". In: *ACM PODC*. 2017, pp. 303–312.

[DLS88] C. Dwork, N. Lynch, and L. Stockmeyer. “Consensus in the Presence of Partial Synchrony”. In: *Journal of the ACM* 35.2 (1988), pp. 288–323.

[DN92] C. Dwork and M. Naor. “Pricing via Processing or Combatting Junk Mail”. In: *Advances in Cryptology (CRYPTO)*. 1992, pp. 139–147.

[Dou02] J. R. Douceur. “The Sybil Attack”. In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 251–260.

[Eth] Ethereum Foundation. *Ethereum Proof-of-Stake Consensus Specifications*. URL: `https://github.com/ethereum/consensus-specs` (visited on 10/12/2026).

[FLP85] M. J. Fischer, N. A. Lynch, and M. S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process”. In: *J. ACM* 32.2 (1985), pp. 374–382.

[GDW19] D. Guo, J. Dong, and K. Wang. "Graph structure and statistical properties of Ethereum transaction relationships". In: *Inf. Sci.* 492 (2019), pp. 58–71.

[Gel+23] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou. "Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing". In: *ACM PPoPP*. 2023, pp. 232–244.

[Ger+16] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *ACM CCS*. 2016, pp. 3–16.

[Gil+17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies". In: *ACM SOSP*. 2017, pp. 51–68.

[GKL15] J. Garay, A. Kiayias, and N. Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *EUROCRYPT*. 2015, pp. 281–310.

[Gud+20] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais. "SoK: Layer-Two Blockchain Protocols". In: *Financial Cryptography and Data Security (FC)*. 2020, pp. 201–226.

[Hei+23] L. Heimbach, L. Kiffer, C. Ferreira Torres, and R. Wattenhofer. "Ethereum's Proposer-Builder Separation: Promises and Realities". In: *ACM IMC*. 2023, pp. 406–420.

[HLZ24] H. Huang, Y. Lin, and Z. Zheng. "Account migration across blockchain shards using fine-tuned lock mechanism". In: *IEEE INFOCOM 2024*. 2024, pp. 271–280.

[HS91] S. Haber and W. S. Stornetta. "How to Time-Stamp a Digital Document". In: *Journal of Cryptology* 3.2 (1991), pp. 99–111.

[Hua+22] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo. "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding". In: *IEEE INFOCOM*. 2022, pp. 1968–1977.

[Kal+18] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten. "Arbitrum: Scalable, private smart contracts". In: *USENIX Security*. 2018, pp. 1353–1370.

[Kia+17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol". In: *CRYPTO*. 2017, pp. 357–388.

[KK98] G. Karypis and V. Kumar. "A fast and high quality multilevel scheme for partitioning irregular graphs". In: *SIAM J. Sci. Comput.* 20.1 (1998), pp. 359–392.

[Kok+18] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford. “OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding”. In: *IEEE S&P*. 2018, pp. 583–598.

[Kró+21] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière. “Shard scheduler: Object placement and migration in sharded account-based blockchains”. In: *ACM AFT*. 2021, pp. 43–56.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *ASIACRYPT*. 2010, pp. 177–194.

[Li+24] P. Li, M. Song, M. Xing, Z. Xiao, Q. Ding, S. Guan, and J. Long. “Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement”. In: *ACM Web Conf*. 2024, pp. 2836–2846.

[Liu+26] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun. "Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception". In: *ACM Web Conf.* 2026, pp. 5493–5502.

[LSP82] L. Lamport, R. Shostak, and M. Pease. "The Byzantine Generals Problem". In: *ACM Trans. Program. Lang. Syst.* 4.3 (1982), pp. 382–401.

[Luu+16] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena. "A Secure Sharding Protocol For Open Blockchains". In: *ACM CCS.* 2016, pp. 17–30.

[Meh+25] B. Mehta, N. Baghel, M. J. Amiri, B. T. Loo, and R. Marcus. "Adaptive Sharding in Untrusted Environments". In: *Proc. ACM Manag. Data* 3.6 (2025), pp. 1–28.

[Mer87] R. C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *Advances in Cryptology (CRYPTO)*. 1987, pp. 369–378.

[MM02] P. Maymounkov and D. Mazières. “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric”. In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 53–65.

[Nag+25] Á. Nagy, J. Tapolcai, I. A. Seres, and B. Ladóczki. “Forking the RANDAO: Manipulating Ethereum’s Distributed Randomness Beacon”. In: *ACM CCS*. 2025, pp. 873–887.

[Nak08] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. `https://bitcoin.org/bitcoin.pdf`. 2008.

[Ngu+19] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai. “OptChain: Optimal Transactions Placement for Scalable Blockchain Sharding”. In: *IEEE ICDCS*. 2019, pp. 525–535.

[NTT21] J. Neu, E. N. Tas, and D. Tse. "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma". In: *IEEE S&P*. 2021, pp. 446–465.

[Pac+23] M. Pacheco, G. Oliva, G. K. Rajbahadur, and A. Hassan. "Is my transaction done yet? an empirical study of transaction processing times in the ethereum blockchain platform". In: *ACM Trans. Softw. Eng. Methodol.* 32.3 (2023), pp. 1–46.

[PAT24] U. Pavloff, Y. Amoussou-Guenou, and S. Tucci-Piergiovanni. "Byzantine attacks exploiting penalties in Ethereum PoS". In: *IEEE/IFIP DSN*. 2024, pp. 53–65.

[Pig+25] M. Pigaglio, O. Ascigil, M. Król, F. Lange, K. Peeroo, S. Rene, R. Sadre, V. Stankovic, and E. Rivière. "PANDAS: Peer-to-peer, Adaptive Networking Allowing Data Availability Sampling within

Ethereum Consensus Timebounds". In: *Middleware*. 2025, pp. 167–179.

[PKR23] M. Pigaglio, M. Król, and E. Rivière. "Exploring locality in Ethereum transactions". In: *IEEE BRAINS*. 2023, pp. 1–8.

[Qi+26] J. Qi, D. Ding, J. Li, J. Cao, Y.-C. Chen, G. Xue, and S. Liu. "BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains". In: *ACM Web Conf.* 2026, pp. 5547–5556.

[Rou24] T. Roughgarden. "Transaction Fee Mechanism Design". In: *J. ACM* (2024), pp. 1–25.

[Sch+22] C. Schwarz-Schilling, J. Neu, B. Monnot, A. Asgaonkar, E. N. Tas, and D. Tse. "Three Attacks on Proof-of-Stake Ethereum". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 560–576.

[Sch90] F. B. Schneider. “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial”. In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[She+21] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath. “BFT Protocol Forensics”. In: *ACM CCS*. 2021, pp. 1722–1743.

[SZ15] Y. Sompolinsky and A. Zohar. “Secure High-Rate Transaction Processing in Bitcoin”. In: *Financial Cryptography and Data Security (FC)*. 2015, pp. 507–527.

[Wan+19] G. Wang, Z. J. Shi, M. Nixon, and S. Han. “SoK: Sharding on Blockchain”. In: *ACM AFT*. 2019, pp. 41–61.

[Woo14] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, `https://ethereum.github.io/yellowpaper/paper.pdf`. 2014.

[WW19] J. Wang and H. Wang. “Monoxide: Scale out blockchains with asynchronous consensus zones”. In: *USENIX NSDI*. 2019, pp. 95–112.

[Yin+19] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. “HotStuff: BFT Consensus with Linearity and Responsiveness”. In: *ACM PODC*. 2019, pp. 347–356.

[ZMR18] M. Zamani, M. Movahedi, and M. Raykova. “RapidChain: Scaling Blockchain via Full Sharding”. In: *ACM CCS*. 2018, pp. 931–948.

[ZPY23] Y. Zhang, S. Pan, and J. Yu. “TxAllo: Dynamic Transaction Allocation in Sharded Blockchain Systems”. In: *IEEE ICDE*. 2023, pp. 721–733.